BÖLÜM 7 / MODULE 7

Klasik MAANG Tasarım Problemleri - Bölüm 1

Classic MAANG System Designs - Part 1

TinyURL (Base62), Pastebin (S3 + TTL), Dağıtık Rate Limiter (Redis Lua), Web Crawler (URL Frontier), Bildirim Servisi ve Proximity/Yelp (Geohash vs QuadTree).

TinyURL (Base62 encoding), Pastebin (S3 + TTL), Distributed Rate Limiter (Redis Lua), Web Crawler (URL Frontier), Notification System, and Yelp Proximity (Geohash vs QuadTree).

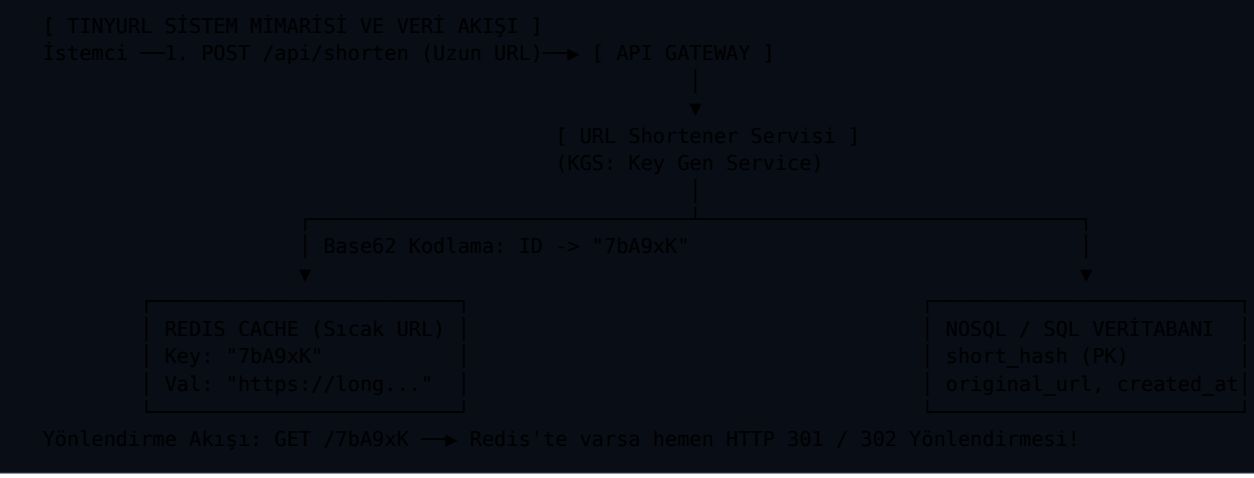
37

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

URL Kısaltıcı Tasarımı (TinyURL / Bit.ly): Base62, Çarpışma Önleme ve 100M DAU

Designing a URL Shortener (TinyURL / Bit.ly): Base62 & Scale

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphanedeki 500 sayfalık kalın bir kitabın adını arkadaşınıza söylerken kitabın tüm içeriğini okumazsınız; ona 'A-12 nolu raftaki kitap' dersiniz (Kısa Kod). Arkadaşınız rafa gider ve koca kitabı bulur. TinyURL de budur: 200 karakterlik devasa bir internet adresini alır (`https://example.com/very/long/url...`), onu 7 karakterlik sevimli bir plakaya (`tinyurl.com/a8B9z1`) dönüştürür. Biri o plakayı tıkladığında sistem plakayı gerçek adrese yönlendirir!

🇬🇧 ENGLISH (ELI5)

Instead of shouting the entire 500-page contents of an encyclopedia across a room, you hand your friend a tiny library call number: 'Shelf B-42' (Short Code). Your friend walks to shelf B-42 and retrieves the tome. TinyURL works identically: it takes an unruly 250-character URL and hashes it into a compact 7-character vanity token (`tiny.one/x9K2pQ`). When a browser accesses that token, the server returns an instant HTTP redirect to the long target destination!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesi TinyURL Tasarım Analizi: 1) Kapasite Hesabı: 100M yeni URL/gün. 10 yılda: 100M x 365 x 10 ≈ 365 Milyar URL. 2) Base62 Kodlama: [0-9, a-z, A-Z] = 62 karakter. 7 karakterlik bir kısa kod: 62^7 = 3.52 × 10^12 (3.5 Trilyon benzersiz kombinasyon sunar; 365 milyarı fazlasıyla karşılar). 3) Hash Çarpışması Önleme: MD5/SHA256 alıp ilk 7 karakteri kesmek çarpışma (Collision) riski taşır. En iyi mimari: 'Key Generation Service' (KGS) ile bağımsız bir servisin önceden benzersiz 7 karakterlik Base62 anahtarları üretip veritabanına koymasıdır. 4) HTTP 301 vs 302: 301 (Moved Permanently) tarayıcı tarafından önbelleğe alınır, sunucu yükünü düşürür ancak tıklama analitiğini (analytics) bozar. 302 (Found) geçicidir; her istek sunucudan geçer, tıklama sayımı kusursuz çalışır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG-Grade Architecture for TinyURL: 1) Capacity: 100M writes/day. 10-year horizon: 365 billion URLs. 2) Base62 Encoding: [0-9, a-z, A-Z] yields 62 alphanumeric permutations. A 7-character token provides 62^7 ≈ 3.52 trillion unique keys. 3) Collision Elimination: Hashing the original URL via SHA-256 and truncating introduces collision loops. Optimal approach: Key Generation Service (KGS) that generates sequential monotonic Snowflake IDs and encodes them to Base62, pre-loading tokens into memory. 4) HTTP 301 vs 302 Redirects: 301 (Moved Permanently) is cached by the client browser, reducing origin QPS while breaking real-time click telemetry. 302 (Found / Temporary Redirect) forces every click through the gateway, capturing user-agent, referer, and IP analytics.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: 64-Bit Sayıyı Base62 Karakter Dizisine Dönüştürücü (URL Shortener Motoru)
public class Base62Encoder {
    private static final String BASE62_ALPHABET =
"0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";

    public static String encode(long number) {
        if (number == 0) return "0";
        var sb = new StringBuilder();
        while (number > 0) {
            int remainder = (int) (number % 62);
            sb.append(BASE62_ALPHABET.charAt(remainder));
            number /= 62;
        }
        return sb.reverse().toString();
    }
}

void main() {
    long uniqueId = 12519284192L; // Snowflake veya DB Identity
    String shortCode = Base62Encoder.encode(uniqueId);
    IO.println("Benzersiz ID: " + uniqueId + " ⟶ Kısa Kod: " + shortCode);
    IO.println("Oluşan URL:   https://tiny.url/" + shortCode);
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'MD5 hash'i alıp ilk 7 karakterini keserim' deyip geçmeyin! İki farklı URL aynı ilk 7 karaktere sahip olabilir (Çarpışma). Mülakatçıya Snowflake ID + Base62 dönüşümünü veya KGS (Key Generation Service) ön-üretim modelini sunduğunuzda doğrudan Staff seviyesine çıkarsınız.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Suggesting a naive SHA-256 truncation without addressing hash collisions is an immediate deduction. Propose a distributed Key Generation Service (KGS) or a monotonic Snowflake sequence mapped deterministically through Base62.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Base62 [0-9, a-z, A-Z] 7 karakterde 3.5 trilyon benzersiz kombinasyon üretir.
- HTTP 302 yönlendirmesi tıklama analitiğini takip etmek için tercih edilir.
- Okuma/Yazma oranı 100:1 olduğu için Redis önbelleği okuma gecikmesini \<5ms seviyesine indirir.

🎯 Key Takeaways for the Interview

- Base62 with 7 characters yields 3.52 trillion collision-free combinations.
- HTTP 302 redirects preserve accurate click telemetry and analytics at the gateway.
- A 100:1 read-to-write ratio mandates an aggressive Redis cache for sub-5ms lookups.

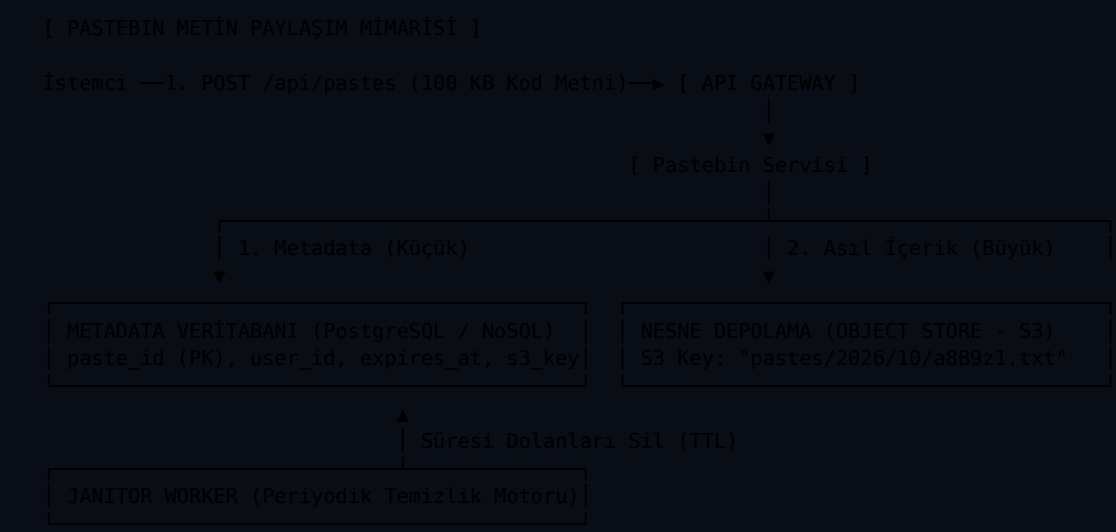
38

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

Pastebin / Metin Paylaşım Servisi Tasarımı: TTL, S3 Blob Depolama ve Temizlik

Designing Pastebin: Object Storage (S3), TTL & Janitor Service

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Büyük bir kütüphanede emanet dolapları olduğunu düşünün: Dolabın kapağında sadece emanet fişi ve son teslim tarihi yazar (Metadata), dolabın içinde ise koca bir bavul eşya durur (Büyük Metin / S3). Siz fiş numarasını verirsiniz, görevli kapağa bakar, bavulunuzu çıkarıp verir. Eğer dolabın süresi dolmuşsa (TTL), gece gelen temizlik görevlisi (Janitor Service) bavulu boşaltıp çöp fırınına atar!

🇬🇧 ENGLISH (ELI5)

Think of a luggage locker terminal at a central train station: The digital locker display shows only the locker ticket ID and expiration date (Metadata), while the spacious locker compartment holds a massive 50-pound travel suitcase (Text Payload / S3). You scan your ticket; the door opens. If a traveler abandons their locker beyond the paid rental window (TTL), a nightly maintenance crew (Janitor Service) empties the locker to free up capacity!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Pastebin ve TinyURL arasındaki en kritik fark: TinyURL'de veri birkaç yüz bayt iken, Pastebin'de her içerik 10 KB - 10 MB arasında devasa metinler içerebilir. 1) Veri Ayrımı (Storage Segregation): Büyük metin gövdesi veritabanında (RDBMS) ASLA `TEXT/BLOB` olarak tutulmaz! Metin doğrudan ucuz ve sınırsız ölçeklenen Nesne Depolamada (AWS S3, MinIO) saklanır. Veritabanında yalnızca metadata tutulur (`paste_id`, `s3_key`, `created_at`, `expires_at`). 2) Süre Aşımı ve Temizlik (TTL & Janitor): Süresi dolan kayıtlar veritabanından anında silinmez (DB kilitlenmesini önlemek için). İstemci okurken `expires_at \< now()` ise '404 Bulunamadı' dönülür. Arka planda çalışan bir 'Janitor Service' (gece batch worker) süresi dolan kayıtları ve S3 dosyalarını toplu olarak temizler.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The critical divergence between TinyURL and Pastebin is payload magnitude: TinyURL stores 200-byte strings, while Pastebin accepts 100KB to 10MB text blobs. Architectural Tenet: Never store unbounded text payloads directly inside relational database tables! Store raw payloads inside cost-effective Object Stores (AWS S3 / Google Cloud Storage), persisting only compact metadata inside PostgreSQL or Cassandra (`paste_id`, `author_id`, `s3_uri`, `expiration_timestamp`). TTL & Lazy Cleanup: Avoid running active delete queries on hot tables. On read paths, check `expires_at \< NOW()` and return 404. Offload actual physical deletions to an asynchronous Janitor Worker that batch-purges expired S3 blobs and DB rows during off-peak hours.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Pastebin Metadata & Süre Kontrol Modeli
import java.time.Instant;

public record PasteMetadata(String pasteId, String s3Uri, Instant expiresAt) {
    public boolean isExpired() {
        return Instant.now().isAfter(expiresAt);
    }
}

void main() {
    var activePaste = new PasteMetadata("pst_101", "s3://pastes/pst_101.txt",
Instant.now().plusSeconds(3600));
    var expiredPaste = new PasteMetadata("pst_102", "s3://pastes/pst_102.txt",
Instant.now().minusSeconds(10));

    IO.println("Paste 101 Süresi Doldu mu? " + activePaste.isExpired()); // false
    IO.println("Paste 102 Süresi Doldu mu? " + expiredPaste.isExpired()); // true -> HTTP 404!
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta '10 MB'lık kod parçalarını PostgreSQL tablosunda VARCHAR sütununda saklarım' derseniz elenirsiniz! Veritabanına büyük metin yazmak disk IO ve bellek önbelleğini şişirir; metin daima S3/Blob depolamada saklanmalıdır.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never propose storing multi-megabyte paste payloads inside an RDBMS table column. This severely fragments database pages and destroys buffer pool caching. Payloads belong in Object Storage (S3).

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Büyük veri yükleri (payloads) Nesne Depolamada (S3), metadata ise veritabanında tutulur.

• Süresi dolan içerikler için tembel kontrol (Lazy Evaluation) ve periyodik Janitor Worker kullanılır.

• Kullanıcıların paste aramaları için metadata veritabanında paste_id ve user_id indekslenir.

🎯 Key Takeaways for the Interview

• Large payloads live in Object Storage (S3); metadata lives in relational/document DBs.

• Combine lazy read-time expiration checks with asynchronous background janitor purging.

• Index author IDs and expiration timestamps to support high-performance user dashboard queries.

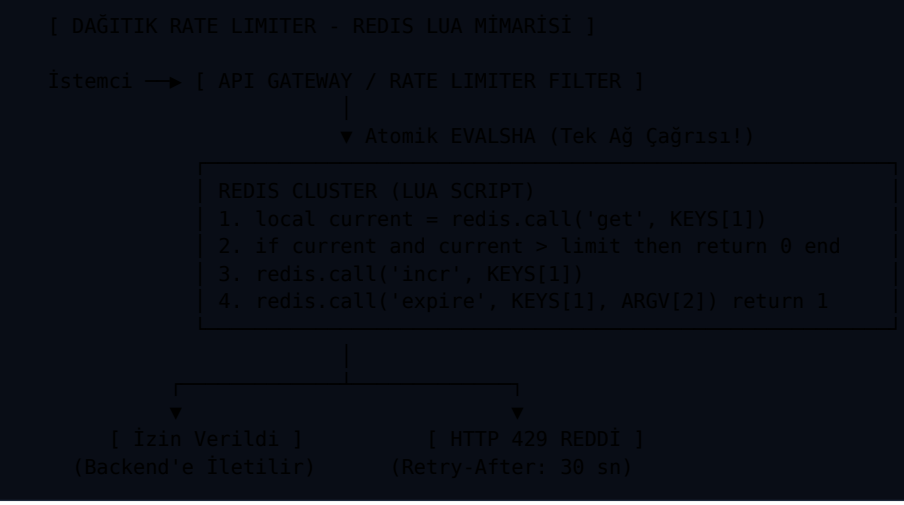
39

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

Dağıtık Hız Sınırlayıcı (Distributed Rate Limiter) Tasarımı: Redis Lua ve Multi-Region

Designing a Distributed Rate Limiter: Redis Lua & Multi-Region

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir gece kulübünün kapısındaki fedaiyi düşünün (Rate Limiter). Kulübün kuralı: 'Bir kişi dakikada en fazla 5 kez içeri girip çıkabilir'. Fedainin elinde telsizle bağlı olduğu merkezi bir liste (Redis) vardır. Bir kişi kapıya geldiğinde fedai listeye bakar, sayacı 1 artırır. Eğer 5 hakkını doldurduysa 'Dur bakalım, 1 dakika sokakta bekle!' der (HTTP 429). Lua Script ise fedainin listeye bakıp sayıyı artırma işini göz açıp kapayıncaya kadar tek bir saniyede (Atomik olarak) yapmasıdır; böylece iki kişi aynı anda kapıya yüklense bile fedaiyi kandıramaz!

🇬🇧 ENGLISH (ELI5)

Picture a VIP nightclub bouncer guarding the velvet rope (Rate Limiter). House rule: 'No patron can enter more than 5 times per minute'. The bouncer holds a radio linked to the central clipboard (Redis). When a patron approaches, the bouncer checks the tally, increments the counter, and if the count exceeds 5, blocks entry with 'Wait 60 seconds on the curb!' (HTTP 429). The Redis Lua script ensures the bouncer checks and increments the clipboard atomically in a single nanosecond, preventing two patrons from rushing the rope simultaneously!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesi Dağıtık Hız Sınırlayıcı Mimarisi: 1) Yarış Durumu (Race Condition) Çözümü: Eğer uygulama Redis'ten `GET` yapıp kodda artırıp `SET` yaparsa, eşzamanlı iki istek aynı değeri okur ve limit aşılır. Çözüm: Redis Lua Scripting. Lua betikleri Redis motorunda tek iş parçacığında atomik olarak çalışır; sıfır kilit maliyetiyle `GET + INCR + EXPIRE` tek adımda biter. 2) Çok Bölgeli (Multi-Region) Senkronizasyon: Kullanıcı hem ABD hem Avrupa bölgesinden istek atıyorsa ne olur? Çözüm: A) Merkezi Redis (Ağ gecikmesi ekler). B) Yerel Redis + Asenkron Sync (CRDT / Eventual Consistency): Her bölgeye kotanın bir kısmı tahsis edilir (Local Quota Allocation) veya asenkron eşleme yapılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Distributed Rate Limiter Architecture: 1) Atomic Race Condition Elimination: Executing a separate `GET`, incrementing in JVM memory, and writing back via `SET` introduces fatal concurrency race windows. Resolution: Redis Lua Scripting executes `GET`, conditional comparison, `INCR`, and `EXPIRE` atomically within the Redis single-threaded engine in a single network round-trip. 2) Multi-Region Synchronization: If a user fires requests simultaneously across US-East and EU-West, cross-datacenter synchronization is required. Architecture Options: A) Centralized primary Redis cluster (adds 100ms cross-Atlantic latency). B) Localized Quota Splitting: Partition a user's 100 req/min allowance into 50 for US and 50 for EU, synchronizing background replenishment asynchronously via batch consensus.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```
// Java 25: Redis Lua Script Tabanlı Rate Limiting Mantığı Simülasyonu
public class RedisLuaRateLimiter {
    public static final String LUA_SCRIPT = """
        local key = KEYS[1]
        local limit = tonumber(ARGV[1])
        local window = tonumber(ARGV[2])

        local current = redis.call('get', key)
        if current and tonumber(current) >= limit then
            return 0 -- İzin Verilmedi (HTTP 429)
        else
            redis.call('incr', key)
            if not current then
                redis.call('expire', key, window)
            end
            return 1 -- İzin Verildi (HTTP 200)
        end
        """;

    public record RateLimitResult(boolean allowed, int remainingLimit, int retryAfterSeconds) {}
}

void main() {
    IO.println("Redis Lua Script Hazırlandı (Atomik GET + INCR + EXPIRE)");
    IO.println("Script Boyutu: " + RedisLuaRateLimiter.LUA_SCRIPT.trim().length() + " karakter");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta rate limiting için istemci IP adresini tek başına anahtar (Key) olarak seçmeyin! Aynı kurumsal ofisteki veya üniversitedeki 10.000 kişi aynı NAT Gateway IP'sini paylaşabilir; tek bir kullanıcının kotası dolunca tüm şirket bloklanır. Anahtar: `UserId` (Giriş yapmışsa) veya `UserId + IP` olmalıdır.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never rely solely on client IP for rate limiting! Behind enterprise NAT proxies or university campuses, 10,000 distinct employees share a single public IP. Key by authenticated `UserId` or a combination of `DeviceFingerprint + IP`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Redis Lua Scripting yarış durumlarını (Race Conditions) sıfır kilit maliyetiyle çözer.
- Kullanıcı deneyimi için HTTP 429 yanıtı ile birlikte 'Retry-After' başlığı dönülmelidir.
- Çok bölgeli sistemlerde yerel kota tahsisi (Local Quota Splitting) ağ gecikmesini engeller.

🎯 Key Takeaways for the Interview

- Redis Lua scripts execute check-and-increment operations atomically without distributed locks.
- Always accompany HTTP 429 responses with accurate Retry-After and X-RateLimit headers.
- Local quota partitioning eliminates cross-datacenter WAN latency on edge rate checks.

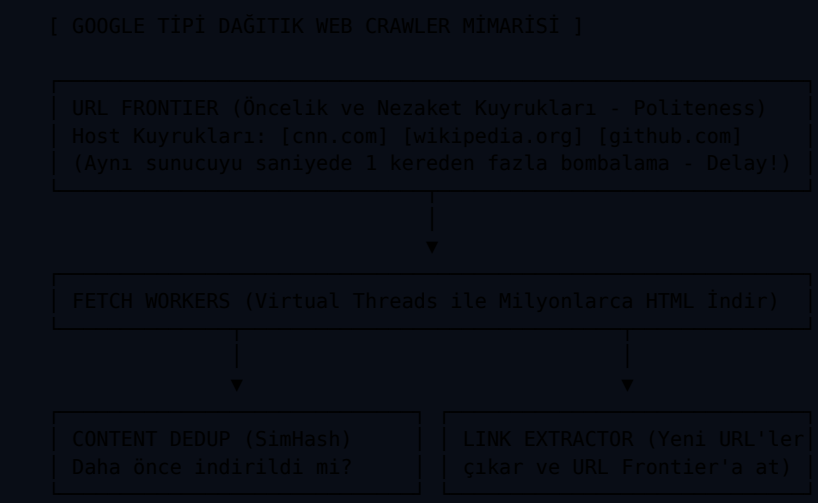
BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

40

Web Gezgini (Web Crawler) Tasarımı: URL Frontier, robots.txt ve Duplication

Designing a Web Crawler: URL Frontier, Politeness & Deduplication

📐 Sistem Mimarisi & Veri Akış Şeması 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir kütüphanedeki tüm kitapları tek tek okuyup fihrist çıkaran çalışkan bir robot düşünün (Web Gezgini / Googlebot). Robot bir sayfa okur, sayfanın altındaki dipnotlarda başka kitapların referansını görür (Link Extraction). O yeni kitapları 'Okunacaklar Listesi'ne (URL Frontier) ekler. Ancak robot çok terbiyelidir (Politeness): Aynı yazarın odasına saniyede 100 kez girip kapıyı çalmaz; her yazarın arasına 2 saniye bekleme süresi koyar. Ayrıca daha önce okuduğu bir kitabı tekrar okumamak için hafızasındaki parmak izlerini (Deduplication) kontrol eder!

🇬🇧 ENGLISH (ELI5)

Imagine an automated librarian cataloging every book ever printed (Googlebot). It opens a book, reads the citations at the bottom (Link Extraction), and writes those new book titles onto a massive 'To-Read Clipboard' (URL Frontier). The librarian is strictly disciplined and polite (Politeness): it never pesters the same author 100 times per second, inserting a courteous 1-second delay between visits. It also checks a fingerprint catalog (Deduplication) so it never rereads identical duplicate books!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Web Crawler Mimarisi: 1) URL Frontier: İndirilecek URL'leri tutan kalıcı kuyruk sistemi. İki temel gereksinim: A) Önceliklendirme (Priority): Kaliteli ve popüler sayfalar önce taranır (PageRank tabanlı). B) Nezaket (Politeness): Aynı hedef alan adına (Host/Domain) giden isteklerin arasına gecikme (ör. 1000ms) konur; hedef sunucu DDoS saldırısına uğramaz. 2) DNS Önbellekleme: Milyonlarca istekte DNS sorguları darboğaz olmasın diye yerel DNS Cache tutulur. 3) İçerik Tekilleştirme (Content Deduplication): İnternetteki içeriğin %30'u mükerrerdir. HTML içeriğinin SimHash veya MinHash parmak izi alınarak Hamming Mesafesi (Hamming Distance) ile benzerlik kontrol edilir. 4) robots.txt: Taranmadan önce sitenin `robots.txt` kuralları okunur ve önbelleğe alınır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Architecture of a Distributed Web Crawler (Googlebot scale): 1) The URL Frontier: Governs URL scheduling via a dual-tiered queueing engine: A) Priority Queues order pages by freshness and PageRank. B) Politeness Queues group URLs by domain hostname, ensuring worker threads enforce delay intervals (e.g., 500ms) to avoid crashing target web hosts. 2) Dedicated Local DNS Caching: DNS lookups represent 30%+ of network latency; crawlers maintain in-memory high-throughput DNS caches. 3) Near-Duplicate Content Elimination: Over 30% of web pages are identical mirrors. Crawlers compute cryptographic 64-bit `SimHash` or `MinHash` locality-sensitive signatures, comparing Hamming Distance to discard duplicates before indexing. 4) robots.txt Compliance: Robots Exclusion Protocol files are fetched and cached in memory prior to scraping target paths.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: SimHash Benzerlik ve Tekilleştirme (Deduplication) Algoritması
import java.math.BigInteger;

public class SimHashDeduplicator {
    public static long computeSimHash(String text) {
        int[] v = new int[64];
        String[] words = text.toLowerCase().split("\\s+");
        for (String word : words) {
            long hash = word.hashCode(); // Basit hash simülasyonu
            for (int i = 0; i < 64; i++) {
                if (((hash >> i) & 1L) == 1L) v[i]++;
                else v[i]--;
            }
        }
        long simHash = 0L;
        for (int i = 0; i < 64; i++) {
            if (v[i] > 0) simHash |= (1L << i);
        }
        return simHash;
    }

    // İki parmak izi arasındaki Hamming Mesafesi (Farklı bit sayısı)
    public static int hammingDistance(long h1, long h2) {
        return Long.bitCount(h1 ^ h2);
    }
}

void main() {
    String doc1 = "Java 25 and Spring Boot system design interview guide";
    String doc2 = "Java 25 and Spring Boot system design interview tutorial"; // Çok benzer!

    long hash1 = SimHashDeduplicator.computeSimHash(doc1);
    long hash2 = SimHashDeduplicator.computeSimHash(doc2);

    int distance = SimHashDeduplicator.hammingDistance(hash1, hash2);
    IO.println("Hamming Mesafesi: " + distance + " bit (Mesafe <= 3 ise MÜKERRER KABUL EDİLİR!)");
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'BFS (Breadth-First Search) ile tararım' deyip geçmeyin! Saf BFS yaparsanız tek bir sitedeki 100.000 linki sırayla indirip o web sitesini çökertebilirsiniz (İhlal: Impolite). Nezaket (Politeness) kuyrukları ile farklı hostların dönüşümlü taranması zorunludur.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never propose naive Breadth-First Search (BFS) for web crawling. A pure BFS traversal hammers a single domain with thousands of concurrent connections, effectively staging a DDoS attack. Explicitly design Politeness queues.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• URL Frontier, önceliklendirme ve nezaket (politeness) dengesini kuran omurgadır.

• SimHash ve MinHash algoritmaları milyarlarca web sayfasındaki benzerlikleri saniyede eler.

• Yerel DNS önbellekleme, web tarama hızını %30 artıran en kritik optimizasyondur.

🎯 Key Takeaways for the Interview

• The URL Frontier enforces both crawl prioritization and domain politeness constraints.

• SimHash algorithms detect near-duplicate pages in constant time using Hamming distance.

• Dedicated in-memory DNS caching eliminates substantial latency bottlenecks during massive crawls.

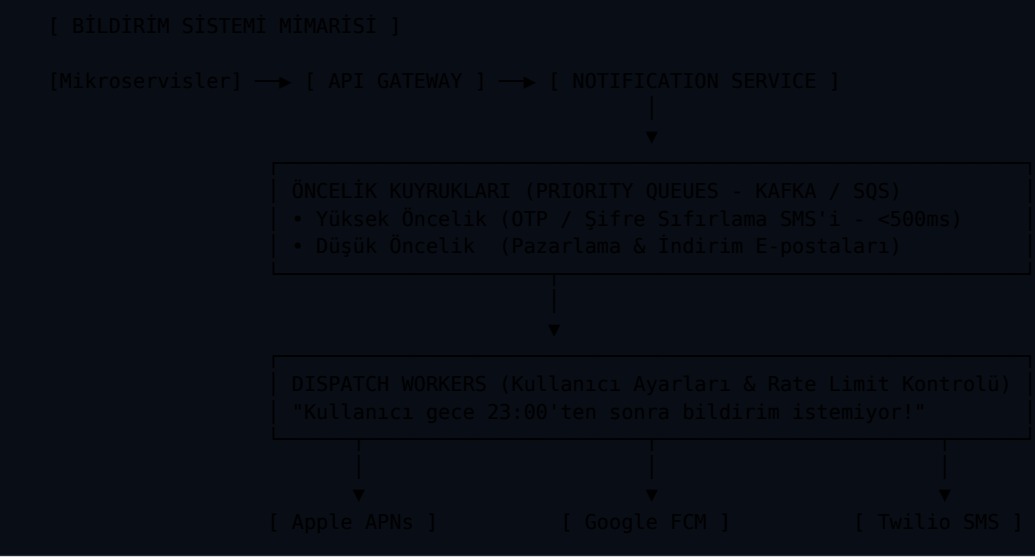
41

BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

Bildirim Sistemi (Notification Service) Tasarımı: Öncelik Kuyrukları ve Rate Limits

Designing a Notification System: Priority Queues & Rate Limits

📐 Sistem Mimarisi & Veri Akış Şeması — 📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu) — 💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirketin kargo ve mektup odasını düşünün: Masada iki sepet vardır. Kırmızı sepet (Yüksek Öncelik): 'Kullanıcının kredi kartı şifresi SMS'i'. Bu hemen 2 saniyede gönderilmelidir! Mavi sepet (Düşük Öncelik): 'Haftasonu indirim bülteni'. Bu 3 saat sonra gitse de kimse ölmez. Görevli çıraklar daima kırmızı sepetteki mektupları önce taşır. Ayrıca görevlinin elinde bir kural listesi vardır: 'Ahmet Bey'e günde 3'ten fazla reklam SMS'i atma, yoksa bizi mahkemeye verir!' Bildirim sistemi işte bu akıllı mektup odasıdır!

🇬🇧 ENGLISH (ELI5)

Picture a corporate dispatch room with two delivery trays. The Red Tray (High Priority) holds emergency 2FA one-time passwords: these MUST land on customer phones within 3 seconds! The Blue Tray (Low Priority) holds weekly marketing coupons: whether these arrive now or at 3:00 AM doesn't matter. Couriers always empty the red tray first. The dispatcher also enforces user opt-outs: 'Never send marketing pushes after 10:00 PM, and cap marketing at 2 pushes per user per day!' That is an enterprise Notification Engine!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesi Bildirim Sistemi Mimarisi: 1) Doğrulama & Kullanıcı Tercihleri (User Preferences): Kullanıcı hangi kanalları açtı? (iOS Push, Android FCM, SMS, E-posta). Kullanıcının 'Rahatsız Etmeyin' (Do Not Disturb - DND) saatleri ve zaman dilimi (Timezone) kontrol edilir. 2) Önceliklendirme Kuyrukları (Priority Queues): OTP ve Güvenlik alarmları yüksek öncelikli Kafka topic'lerine; bültenler düşük öncelikli kuyruklara aktarılır. Düşük öncelikli yoğunluk yüksek öncelikli SMS'leri asla bloklayamaz. 3) Hız Sınırlama & Birleştirme (Rate Limiting & De-duplication): Kullanıcıya 1 saatte en fazla 1 SMS gitmesi kuralı Redis ile denetlenir. Benzer bildirimler birleştirilir (Notification Aggregation, ör. 'Ali ve 5 kişi fotoğrafını beğendi'). 4) Üçüncü Parti Sağlayıcılar: Apple APNs, Google FCM, Twilio SMS, SendGrid E-posta entegrasyonları.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Notification System Blueprint: 1) Preference & Consent Engine: Evaluates user settings (APNs vs FCM vs Twilio SMS vs SendGrid), opt-out flags, and local timezone Do Not Disturb (DND) windows. 2) Strict Queue Isolation: Segregates critical transactional OTPs into dedicated high-priority Kafka topics, keeping them isolated from marketing campaign backlogs. 3) Frequency Capping & Aggregation: Redis counters enforce frequency caps (e.g. max 3 promotional pushes/day). Aggregation workers batch high-velocity social notifications (e.g. 'Alice and 14 others liked your post') to avoid notification fatigue. 4) Resilient Downstream Adapters: Wrap third-party gateway integrations (Twilio, APNs) in Circuit Breakers with dead-letter fallback queues.

⚡ Modern Java 25 Uygulaması (Loom & Records)

⚡ Java 25 Implementation

```java
// Java 25: Öncelikli Bildirim Yönlendirme Modeli
public class NotificationRouter {
    public enum Priority { HIGH_OTP, LOW_MARKETING }
    public record Notification(String userId, String channel, String message, Priority priority) {}

    public String routeToQueue(Notification notif) {
        return switch (notif.priority()) {
            case HIGH_OTP -> "QUEUE_CRITICAL_HIGH_SPEED (SLA < 1s)";
            case LOW_MARKETING -> "QUEUE_BULK_BATCH (SLA < 4 hours)";
        };
    }
}

void main() {
    var router = new NotificationRouter();
    var otp = new NotificationRouter.Notification("u1", "SMS", "Kodunuz: 9481",
NotificationRouter.Priority.HIGH_OTP);
    var promo = new NotificationRouter.Notification("u2", "PUSH", "%50 İndirim Başladı!",
NotificationRouter.Priority.LOW_MARKETING);

    IO.println("OTP Yolu:   " + router.routeToQueue(otp));
    IO.println("Promo Yolu: " + router.routeToQueue(promo));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Tüm bildirimleri tek bir RabbitMQ kuyruğuna atarım' demeyin! 10 milyon kullanıcılı bir e-posta kampanyası başladığında kuyruk şişer ve banka şifresi bekleyen bir kullanıcının SMS'i 45 dakika sonra gider. Kritik OTP'ler ile pazarlama bildirimleri KESİNLİKLE ayrı kuyruklarda izole edilmelidir.

⚠️ Interview Trap & Bottleneck Alert

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Funneling all notifications into a single shared queue is an architectural catastrophe. A 5-million marketing campaign will choke the queue, delaying password reset OTPs by 30 minutes! High-priority OTPs require dedicated, isolated queue infrastructure.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Kritik OTP ve güvenlik bildirimleri pazarlama kuyruklarından tamamen izole edilir.
- Kullanıcı tercihleri, zaman dilimi ve DND saatleri bildirim fırlatılmadan önce denetlenir.
- Benzer bildirimler (Aggregation) birleştirilerek kullanıcının spam hissetmesi engellenir.

🎯 Key Takeaways for the Interview

- Isolate transactional OTP alerts from marketing campaigns into dedicated queues.
- Filter notifications against user opt-outs, timezones, and frequency caps before dispatch.
- Notification aggregation batches high-frequency interactions into clean digests.

42

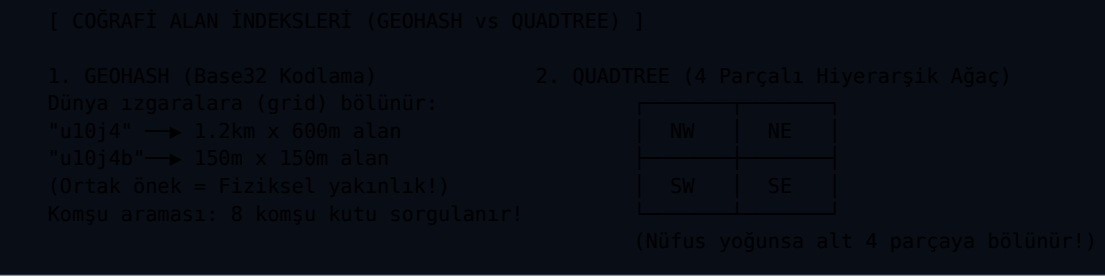
BÖLÜM: KLASIK MAANG TASARIM PROBLEMLERI - BÖLÜM 1 | MODULE: CLASSIC MAANG SYSTEM DESIGNS - PART 1

Yakındaki Yerler / Mekan Arama Servisi (Proximity / Yelp): Geohash vs QuadTree vs Google S2

Designing a Proximity Service (Yelp / Google Maps): Geohash vs QuadTree

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Haritada 'Bana en yakın 10 kahveciyi bul' dediğinizde, dünyadaki 500 milyon kahvecinin koordinatını tek tek Pisagor teoremiyle ($x^2 + y^2$) hesaplamaya çalışırsanız sunucu patlar! Bunun yerine dünya bir satranç tahtası gibi karelere bölünür (Geohash). Her karenin posta kodu gibi bir adı vardır (ör. 'u10j4'). Siz o karenin içindeyseniz, sistem sadece o karenin ve etrafındaki 8 komşu karenin içindeki kahvecilere bakar. 500 milyon dükkan yerine sadece 50 dükkana bakarak 1 milisaniyede sonucu bulur!

🇬🇧 ENGLISH (ELI5)

Searching 'Find the 10 closest cafes near me' cannot compute the Pythagorean distance formula across all 500 million cafes worldwide on every GPS ping! Instead, divide the planet into an indexed grid of boxes (Geohash). Each grid box gets an alphanumeric postal code (e.g. `u10j4`). When you query from that box, the system simply queries that box plus its 8 immediate neighboring boxes. The search space shrinks from 500 million points down to 50 in 1 millisecond!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

MAANG Seviyesi Mekan Arama (Proximity) Mimarisi: 1) Neden Standart SQL İndeksi Yetmez? Enlem (Latitude) ve Boylam (Longitude) iki boyutlu veridir; standart B-Tree indeksinde `WHERE lat BETWEEN ... AND lon BETWEEN ...` sorgusu indeks birleştirmesi (Index Merge) yüzünden çok yavaştır. 2) Geohash: Enlem ve boylamı iç içe geçirerek Base32 karakter dizisine kodlar (1D string). Özellik: Ortak önek (Prefix) paylaşan noktalar fiziksel olarak birbirine yakındır (Örn: `u10j4` 6 karakter ≈ 1.2 km). Kenar etkisi (Edge Effect) için merkez kutu + 8 komşu kutu sorgulanır. Redis `GEOADD` ve `GEORADIUS` komutları arka planda Geohash ve Sorted Set kullanır. 3) QuadTree: Her düğümün en fazla 4 çocuk düğümü olan hiyerarşik ağaçtır. Düğümdeki mekan sayısı eşiği (ör. 100 dükkan) aşınca o alan 4 alt parçaya bölünür (Yoğun şehir merkezlerinde derin, okyanusta sığ ağaç). 4) Google S2: Dünyayı küp projeksiyonu ve Hilbert Eğrisi ile 64-bit tam sayılara böler; Google Haritalar'ın standardıdır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

MAANG Proximity Service Architecture: 1) The 2D Spatial Indexing Challenge: Traditional B-Trees index single dimensions efficiently; evaluating `WHERE lat BETWEEN x AND y AND lon BETWEEN a AND b` degrades to full table filtering. 2) Geohash: Interleaves binary bits of latitude and longitude into a Base32 string. Key Trait: Strings sharing prefixes share spatial proximity (`u10j4` has 1.2km precision). The Boundary Problem: Two adjacent points across a boundary can share completely different prefixes; solved by querying the central box plus its 8 surrounding neighbor cells. Redis native geospatial commands (`GEOADD`, `GEOSEARCH`) leverage 52-bit Geohashes within Sorted Sets. 3) QuadTree: In-memory 2D spatial tree where each quadrant recursively splits into 4 sub-quadrants when point density exceeds a threshold (e.g., 100 POIs). Deeply sub-divided in Manhattan; undivided across deserts. 4) Google S2 Geometry: Projects the spherical earth onto a cube using Space-Filling Hilbert Curves into 64-bit integers; de facto standard for Uber and Google Maps.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```java
// Java 25: Basitleştirilmiş Geohash Önek Eşleme ve Komşu Filtreleme
import java.util.*;

public class GeohashProximityEngine {
    public record Place(String name, String geohash6) {}

    private final Map<String, List<Place>> geohashIndex = new HashMap<>();

    public void indexPlace(Place place) {
        geohashIndex.computeIfAbsent(place.geohash6(), k -> new ArrayList<>()).add(place);
    }

    public List<Place> searchNearby(String userGeohash5Prefix) {
        List<Place> results = new ArrayList<>();
        // 5 karakterlik önek (~4.9 km x 4.9 km alan) ile başlayan tüm yerleri topla
        for (var entry : geohashIndex.entrySet()) {
            if (entry.getKey().startsWith(userGeohash5Prefix)) {
                results.addAll(entry.getValue());
            }
        }
        return results;
    }
}

void main() {
    var engine = new GeohashProximityEngine();
    engine.indexPlace(new GeohashProximityEngine.Place("Kadıköy Kahvecisi", "sxk97a"));
    engine.indexPlace(new GeohashProximityEngine.Place("Moda Dondurmacısı", "sxk97b"));
    engine.indexPlace(new GeohashProximityEngine.Place("Ankara Cafe", "sx5f2a"));

    IO.println("Kadıköy (sxk97) Civarındaki Mekanlar:");
    for (var p : engine.searchNearby("sxk97")) {
        IO.println("• " + p.name() + " [Geohash: " + p.geohash6() + "]");
    }
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta mekan bilgilerinin (isim, adres, menü) QuadTree veya Redis'in içinde saklanacağını söylemeyin! Mekan verisi (metadata) PostgreSQL/MySQL'de tutulur; QuadTree veya Geohash indeksinde SADECE `Place_ID` ve `Geohash` tutulmalıdır. Aksi halde önbellek belleği çöker.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Do not pack heavy business payloads (reviews, menu photos, descriptions) inside the spatial QuadTree/Geohash index. The spatial index must hold only compact `(Place_ID, Coordinates)` tuples; full metadata is resolved from the primary DB.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Geohash 2D koordinatları tek boyutlu string'lere dönüştürerek hızlı arama sağlar.

• Kenar etkisi (Boundary Effect) problemini çözmek için merkez hücre + 8 komşu taranır.

• QuadTree bellek içi yoğunluğa göre dinamik olarak alt parçalara bölünür.

🎯 Key Takeaways for the Interview

• Geohash flattens 2D coordinates into 1D strings, enabling standard index range queries.

• Mitigate Geohash boundary cliffs by querying the central cell plus its 8 adjacent neighbors.

• QuadTrees adapt dynamically to population density, allocating deep nodes only in dense cities.